



模板类型

▼ 函数模板

在函数中使用模板，即函数模板。

```
template <typename T>
const T& max(const T &a, const T &b)
{
    std::cout << "max(&, &) = ";
    return a > b ? a : b;
}
```

C++

函数模板可以自动推导模板参数类型，根据传入的参数自动确定类型。

在函数模板中，参数类型是根据传入的实际参数进行推导的，函数参数有明确的类型，因此推导过程是相对简单的。

所以函数模板大多数情况下不需要显式声明模板参数，即在调用时可以省略<>参数。

但在只有返回值使用模板参数的函数中，编译器无法根据形参推断模板参数类型，所以此时需要显式声明模板参数。

```
template <typename T>
T func(int a) {
    return static_cast<T>(a);
}

int main() {
    int result1 = func<int>(10);          // 指定返回值为 int
    double result2 = func<double>(10);    // 指定返回值为 double
    return 0;
}
```

C++

▼ 类模板

在类中使用模板，即模板类。模板类本质是模板而不是类，对模板进行实例化后可以生成对应类型的类定义。

类模板通常有多个成员变量或方法，这些成员的类型可能互相关联，也可能没有明确的外部信息来推导出模板参数。

C++ 是一种**静态类型语言**，所有类型信息必须在**编译时确定**。这意味着，在模板类实例化时，编译器需要知道模板的确切类型以便生成代码。而由于类的复杂性，编译器很难自动推导得到模板应该是什么类型。

如果模板类允许自动推导类型，可能会导致推导失败或出现二义性。特别是在模板类中有多个模板参数的情况下，推导机制可能无法确定唯一的类型或产生歧义，从而导致编译错误或不可预测的行为。

所以，**模板类没有自动推导**，使用时必须显式指定模板参数类型。

▼ 使用例

C++

```
#include <iostream>
class normal_class {
public:
    normal_class(int e) :element(e) {}
    int get_element() {
        return element;
    }
private:
    int element;
};
template <typename T>
class template_class {
public:
    template_class(T e) :element(e) {}
    T get_element() {
        return element;
    }
private:
    T element;
};
int main() {
    normal_class a(1);
    template_class<int> b(2);
    template_class<double> c(3.14);
    std::cout << a.get_element() << b.get_element() << c.get_element() << std::endl;
}
```

普通类可以进行分离编译，是因为其定义在编译阶段已经被确定并分配内存空间，可以在链接时合并。

而模板类本身不是类而是模板，其定义一开始并不确定，在开始编译后，**确定具体类型才可以将定义进行实例化并分配内存空间**。

所以使用模板类的源代码与模板类的定义必须在同一编译单元，这样模板类才能根据使用时的具体类型进行定义实例化，所以模板类定义必须放在头文件参与其他的编译单元。

如果模板类的定义在源文件，那么便会独自作为一个编译单元，直到链接阶段合并，而在编译阶段是无法对一个未实例化的模板进行编译的。

▼ 变量模板

模板变量是 C++14 引入的语言特性，它允许我们定义依赖模板参数的变量。

变量模板是一个用于构造某一类型对象的模板，所以其必须接受一个对象，而不能是一个函数或者一个类，使用时需要注意：

- 可以接受某个函数的返回值作为对象，但不能接受函数本身；
- 可以接受 lambda 对象，不过需要注意的是 lambda 对象本身还是其返回值；
- 不能接受一个类，但可以接受类初始化的实例。

变量模板既可以使用类型参数也可以使用非类型参数，不过由于变量模板用于返回一个具体的实例对象，所以大多数情况下均使用的是非类型参数，如：

▼ 非类型参数示例

C++

```
template<std::size_t N>
constexpr std::size_t factorial = N * factorial<N - 1>;

// 特化
template<>
constexpr std::size_t factorial<0> = 1;

// 用法
constexpr auto f5 = factorial<5>; // 120
```

▼ lambda 对象的变量模板

C++

```
// lambda对象本身作为变量
template<int N>
inline constexpr auto multiplier = []() { return N * N; };

int result = multiplier<3>(); // result是一个lambda对象，需要执行

// lambda对象的返回值作为变量
template<int N>
inline constexpr auto multiplier = []() { return N * N; }();

// 用法
int result = multiplier<3>; // multiplier<3>是一个整型，result是对其的拷贝
```

与函数模板和类模板类似，编译器只会实例化编译过程中实际用到的类型，不同的是，类模板实例化后仅生成类型定义，函数模板则生成具体函数代码，它们本身不会占据运行时内存。

而变量模板在实例化时，会立即生成一个具体的具名变量，并实际占据内存空间。

如果代码中多次使用同一类型的变量模板，也只会首次使用时实例化一次，后续使用只是对该变量的引用或拷贝。

▼ 多次使用同一类型的变量模板

C++

```
template<typename T, std::size_t N>
constexpr std::array<T, N> id_array = []{
    std::array<T, N> arr{};
    std::cout << "lambda executed\n"; // 不允许在 constexpr 中输出，仅用于演示
    for (std::size_t i = 0; i < N; ++i)
        arr[i] = i;
    return arr;
}();

int main() {
    auto a = id_array<int, 10>;
    auto b = id_array<int, 10>;
}
```

在本例中，`id_array<int, 10>` 本身就是一个具名变量，`a` 与 `b` 是对其的拷贝。

注意事项

- 变量模板只能定义为**静态存储期**，不可以是非静态成员。
- 变量模板不是函数，不能重载或使用参数推导（必须显式写出模板参数）。
- 如果定义在头文件中，建议使用 `inline constexpr`，以防止链接错误。

▼ 别名模板

别名模板是 C++11 引入的一种语法，用于给一类模板类型起别名。

可以把它看作是模板版的 `typedef`，但功能更强，因为 `typedef` 不能对模板参数进行抽象。

```
// ❌ 错误：typedef 不支持模板
typedef T& Ref<T>; // 错误
```

C++

```
template <typename T>
using Ref = T&; // ✅ 正确

Ref<int> x = some_int; // 等价于 int& x = some_int;
```

C++

别名模板有以下几种常用用途：

- 简化复杂类型

```
template <typename T>
using Vec = std::vector<T, MyAllocator<T>>;

Vec<int> v; // 相当于 std::vector<int, MyAllocator<int>>
```

C++

- 统一接口封装

C++

```
template <typename T>
using Ptr = std::shared_ptr<T>;

Ptr<MyClass> obj; // 等价于 std::shared_ptr<MyClass>
```

- 用于 `concept` 要求中

```
template <typename T>
using Ref = T&;

template <typename T>
concept HasAlias = requires {
    typename Ref<T>; // 要求 Ref<T> 是合法的类型别名
};
```

C++